

FoveaMap — Architecture Vision

Sep 30, 2026 · @Ammar Hashmi

1. Vision and principles

FoveaMap is a four-stage streaming pipeline (ingest, segment, project, publish) whose core asset is a tiered 2.5D grid that stores about 318,000 cells for a 100 m radius instead of 16 million, so the map fits in about 5 MB and updates every frame.

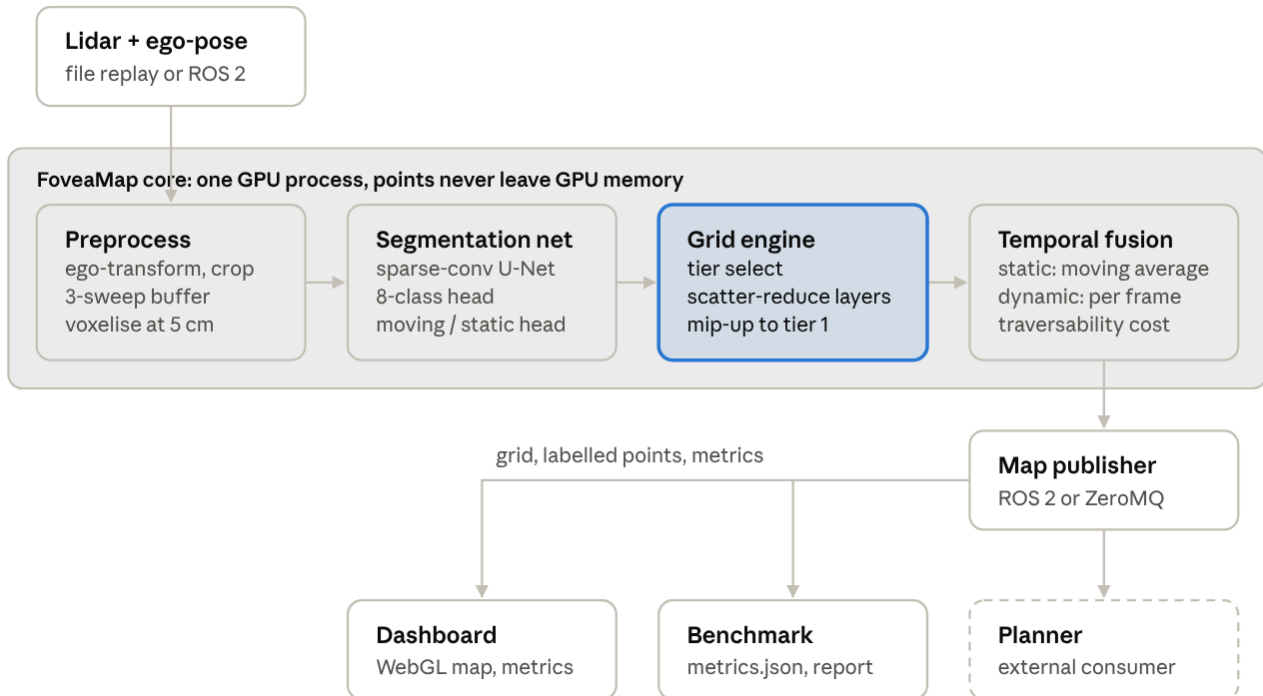
Principles

1. **Resolution follows information.** Cell size grows with range because Lidar point density falls with range. Fine cells where points are dense and decisions urgent; coarse cells where fine cells would be empty.
2. **Exact nesting, no resampling.** Every tier's cell size is an integer multiple of the one inside it, and all tiers share one world-aligned origin. A coarse cell is always an exact union of fine cells, so a point maps to exactly one cell in exactly one tier.
3. **Classify in 3D, decide in 2.5D.** Semantics are learned on points, where geometry is richest; the planner-facing product is a compact per-cell summary.
4. **Static and dynamic never share a layer.** Static layers integrate over time; the dynamic layer is rebuilt every frame so moving objects never leave ghost trails.
5. **Measure everything, per frame.** Latency per stage, bytes per layer and accuracy per distance band are first-class outputs, not afterthoughts.
6. **Swappable parts behind stable interfaces.** Backbone, grid profile and visualiser change by config; the map message format does not.

2. System context and pipeline

Each sweep flows through four GPU stages inside one process, then fans out to the dashboard, the benchmark recorder and any external planner.

Points are classified in 3D, then binned into the foveated grid on the GPU



end-to-end pipeline · 4 core stages, 3 consumers

The grid engine (blue) is the novel part; preprocessing, the network and fusion follow established practice. The dashed planner is outside v1 scope and only consumes the published map.

3. Ingestion and preprocessing

Preprocessing runs on the GPU and targets 4 ms per sweep; its job is to hand the network a clean, ego-aligned, range-cropped point set.

1. **Decode.** Adapters for KITTI `.bin`, nuScenes, PCD and ROS 2 `PointCloud2` emit one tensor: x, y, z, intensity, relative time, ring index.
2. **Ego-motion.** Transform into the vehicle frame at sweep end using the supplied pose; de-skew per point when timestamps exist (FR-2).
3. **Crop and filter.** Drop points beyond 100 m horizontal range, below -3 m or above +5 m relative to the sensor, and self-hits on the vehicle body box.
4. **Sweep buffer.** Keep the last $K = 3$ ego-aligned sweeps in a GPU ring buffer; they feed the motion cue in the network.
5. **Distance tagging.** Compute each point's Chebyshev distance $\max(|x|, |y|)$ once; it decides the tier in section 5 and the distance band in evaluation.
6. **Input voxelisation.** Quantise to a 5 cm input voxel grid for the sparse network, keeping a point-to-voxel index so voxel predictions map back to every original point.

4. Perception model

The default model is a sparse 3D convolutional U-Net with two heads (semantic class and moving/static), because sparse convolution scales with occupied voxels, not scene volume, and fits the 25 ms inference budget.

Option	How it sees the scene	Strength	Weakness	Role
Sparse conv U-Net (MinkUNet / SPVCNN style)	Occupied voxels only, 3D convolutions	Accurate, fast on GPU, handles any density	Needs spconv or TorchSparse build	Default
PointNet++	Raw points, hierarchical set abstraction	Simple, no voxel library, named in the brief	Neighbour search slows it on 100k+ points	Baseline + fallback
Range-image CNN (RangeNet++ / SalsaNext style)	2D projection of the spinning sensor	Fastest; plain 2D convs, easy TensorRT	Boundary blur, back-projection errors	Low-power fallback
Point Transformer family	Attention over serialised points	State-of-the-art accuracy	Heavier; harder to hit 25 ms	Research stretch

Network design

- **Input features per voxel:** x, y, z offsets, intensity, range, and relative sweep time for points from the last 3 sweeps. The time channel lets the network see what moved.
- **Encoder-decoder:** 4 down-sampling stages (5 → 10 → 20 → 40 cm effective voxel), skip connections, channel widths 32–256.
- **Head A, semantics:** 8 FoveaMap classes (PRD section 8 taxonomy) plus an ignore label; softmax gives per-point confidence.
- **Head B, motion:** one logit per point, moving vs static, applied to vehicle and person classes and passed to the dynamic layer.

Training

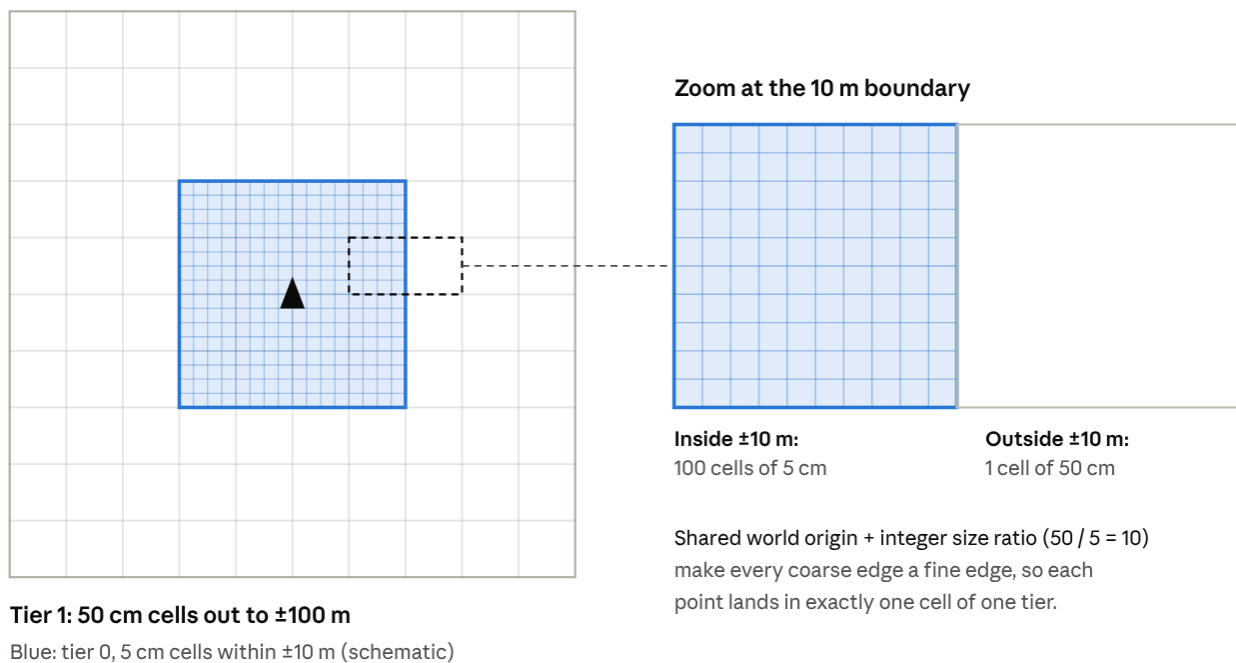
- **Loss:** cross-entropy + Lovász-softmax for semantics; weighted binary cross-entropy for motion (moving points are rare).
- **Range-aware weighting:** far points are few, so the loss up-weights them by distance band; otherwise the model optimises the near field only and accuracy falls off a cliff past 50 m.

- **Augmentation:** random rotation about z, flip, scaling $\pm 5\%$, point dropout that mimics sparser far returns, and instance paste of rare classes (cyclists, pedestrians).
- **Precision:** mixed precision training; FP16 inference exported through ONNX to TensorRT (FR-20).

5. Variable-resolution grid engine

The grid is a stack of concentric square tiers, each a dense, world-aligned 2D array whose cell size is an integer multiple of the tier inside it; that single rule removes alignment errors and point loss at boundaries.

A 50 cm cell is always exactly 10×10 fine cells, so tiers never misalign



foveated grid · 2-tier spec profile, schematic

Tiers are square (Chebyshev distance) rather than circular because square rings tile a Cartesian lattice exactly; a circle would cut cells in half at every boundary.

Tier profiles (16 bytes per cell, 100 m range)

Profile	Tiers	Cells	Map memory
Spec (default)	5 cm to ± 10 m; 50 cm to ± 100 m	318,400	5.1 MB
Graded	5 cm to ± 10 m; 10 cm to ± 25 m; 50 cm to ± 100 m	520,000	8.3 MB

Profile	Tiers	Cells	Map memory
Uniform baseline	5 cm everywhere to ± 100 m	16,000,000	256 MB

The graded profile softens the 10x jump at 10 m (ratios 2 and 5 stay integer). It is the fallback if the boundary shows seams in testing.

Data structure

- **Storage:** one dense array per tier, structure-of-arrays (one contiguous tensor per layer) so GPU kernels read coalesced memory. Each tier is stored as a full square; the inner hole it leaves unused is only 1% of the outer tier's cells.
- **Persistent cell record, 16 B:** $z_{\min}$, $z_{\max}$, z_{mean} , roughness ($4 \times \text{FP16}$); point count (u16); dominant class and confidence ($2 \times \text{u8}$); dynamic flags (u8); traversability cost (u8); overhang clearance in 2 cm steps (u8); age in frames (u8).
- **Per-frame scratch:** an 8-class score histogram per cell (FP16), reused every frame and never stored.
- **Indexing:** world cell index $i = \text{floor}(x / s)$; array slot $i \bmod N$ (a ring buffer). As the vehicle moves, only newly exposed rows and columns are cleared, so the map scrolls instead of being rebuilt (FR-11).
- **Boundary snapping:** the tier-0 box is snapped to the 50 cm lattice, so it moves in whole coarse cells and never splits one.
- **Mip-level fill:** fine cells also aggregate upward into the coarse tier beneath them, so tier 1 is complete everywhere. A planner that wants one resolution reads tier 1 alone; no data is lost.

6. 3D to 2.5D projection and layer fusion

Projection is one GPU pass: each classified point computes its tier and cell key, then scatter-reduce kernels aggregate every layer in parallel, targeting 5 ms per frame.

1. **Tier select.** tier = 0 if $\max(|x|, |y|)$ is inside the snapped tier-0 box, else 1 (else 2 in the graded profile).
2. **Cell key.** $(\text{tier}, \text{floor}(x / s_{\text{tier}}), \text{floor}(y / s_{\text{tier}}))$, packed into one 64-bit integer.
3. **Scatter-reduce.** Atomic min, max and sum over z ; count; per-class score sums weighted by softmax confidence.
4. **Finalise.** Per cell: mean and variance of z , dominant class = argmax of score sums, confidence = its share of the total.

5. **Mip-up.** Tier-0 cells fold into their parent tier-1 cell with the same reductions (min of mins, sum of counts), so tier 1 is complete.

Aggregation rules

Layer	Rule	Why
Ground height	Lowest 10th percentile of ground-class points (fallback: $z_{\min}$)	Robust to grass and noise; keeps curbs and pothole floors
Obstacle height	$z_{\max}$ of non-ground points	Conservative for collision checks
Overhang clearance	Lowest non-ground point above ground + 0.3 m, minus ground height	Detects branches, barriers, truck beds (PRD US-3)
Roughness	Standard deviation of ground z	Separates smooth road from gravel and grass
Step height	Max ground-height difference to the 4 neighbours	Curbs and potholes; compared across tiers using the mip-level so the 10 m edge shows no false step
Class	Confidence-weighted vote	A few confident points beat many uncertain ones
Dynamic	Any point with moving probability > 0.5 and class vehicle or person	Recall first for safety

Traversability cost (0–255). A weighted sum: class prior (road 0, sidewalk 80, vegetation 150, obstacle 255), slope, step height, roughness, and a penalty for low confidence. Cells with fewer than 3 points in the last 5 frames are marked unknown (cost 200), never free.

No point lost. Each frame asserts that the sum of all tier-0 and tier-1 counts (before mip-up) equals the number of cropped input points; a mismatch fails the frame in debug builds.

7. Temporal fusion and dynamic objects

Static layers accumulate evidence across frames; the dynamic layer is rebuilt from scratch every frame, so a pedestrian who walks away leaves no ghost.

- **Static layers.** Each cell's class confidence is updated with an exponential moving average ($\alpha = 0.3$ near, 0.5 far, because far cells get fewer points per frame). Heights take the confidence-weighted mean of the old and new values.

- **Free-space clearing.** A static obstacle cell seen as empty for 3 consecutive frames, with a Lidar ray passing through it, decays toward unknown. This handles parked cars that leave.
- **Dynamic layer.** Contains only this frame's moving vehicle and person points, with a per-cell velocity estimate from the point offset between sweeps (v1: coarse; v2: from a tracker).
- **Stale cells.** Age increments each frame a cell gets no points. After 20 frames (2 s) the cell's confidence halves; the dashboard shows age as a fade.
- **Sensor gaps.** A missed sweep keeps the last map with an incremented age rather than publishing an empty one (PRD NFR-9).

8. Runtime, concurrency and deployment

Perception and grid engine share one GPU process so points never leave device memory; everything else talks to them through messages.

Stage	Runs on	Budget (ms)
Decode, ego-transform, crop	GPU	4
Input voxelisation	GPU	3
Network inference (FP16)	GPU	25
Projection + layer reduction	GPU	5
Temporal fusion + cost	GPU	3
Serialise + publish	CPU	3
Total		43

These are design budgets to be replaced by measured numbers in week 1 and week 8; 43 ms leaves 7 ms headroom under the 50 ms target.

- **Pipelining.** Two CUDA streams overlap frame N's inference with frame N+1's preprocessing, which lifts throughput above 1 / latency.
- **Process layout.** (1) Sensor or replay driver; (2) FoveaMap core (preprocess, model, grid, fusion) in one Python/C++ process; (3) Dashboard server; (4) Benchmark recorder. Core publishes; nobody can block it.
- **Interfaces.** ROS 2 topics: `/foveamap/grid` (custom message: tier headers + layer arrays), `/foveamap/points_labeled`, `/foveamap/metrics`. Without ROS, the same messages go over ZeroMQ, so the demo runs on a plain laptop.

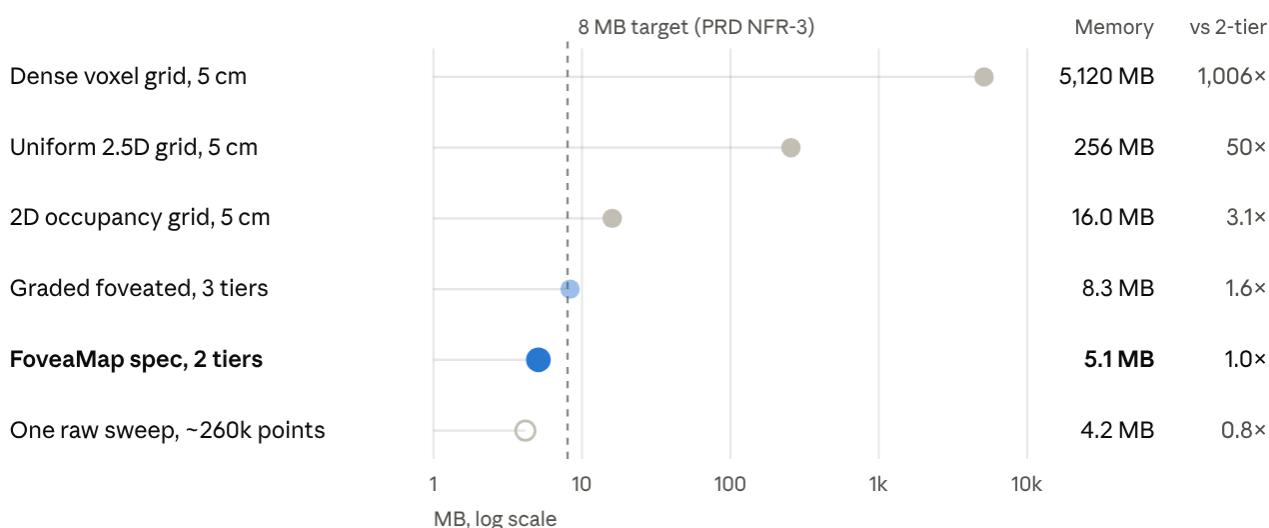
- **Dashboard feed.** A WebSocket bridge sends compressed layer arrays at up to 20 Hz; the browser renders with WebGL. If the browser falls behind it drops frames, never the core.
- **Packaging.** One Docker image pinned to CUDA, PyTorch and the sparse-conv library; `make demo` and `make bench` are the only entry points.

9. Memory and latency budget

The 2-tier map holds 318,400 cells in about 5.1 MB, roughly the size of one raw Lidar sweep, against 256 MB for a uniform 5 cm 2.5D grid and about 5 GB for a dense 5 cm voxel grid.

The 2-tier foveated map needs 5.1 MB, 50× less than a uniform 5 cm 2.5D grid

Map memory for a ± 100 m area, log scale. 16 B per 2.5D cell, 2 B per voxel (8 m tall), 1 B per occupancy cell.



theoretical: cells $\times$ bytes per cell, ± 100 m square; raw sweep assumes $\sim 260k$ points $\times$ 16 B

The graded profile sits just above the 8 MB target, inside the 16 MB hard limit. A sparse voxel hash would shrink the voxel figure a lot, which is why the benchmark measures allocated bytes of a real sparse implementation, not only these dense numbers. The latency budget is in section 8.

10. Evaluation and observability

One harness produces every number the dashboard and the final report show, so demo and report can never disagree.

- **Per frame (always on):** stage timings with GPU synchronisation, allocated bytes per layer and tier, points in vs points binned, cells updated, unknown-cell share.
- **Per sequence (bench mode):** p50 / p95 / p99 latency, sustained FPS, peak VRAM; point mIoU and grid mIoU per class and per distance band (0–10, 10–25, 25–50, 50–100 m); moving-object IoU; curb recall within 10 m.

- **Baselines, same run:** uniform 5 cm 2.5D grid, uniform 5 cm voxel grid (sparse hash, bytes measured), 2D occupancy grid. Same points, same labels, so the comparison is fair.
- **Outputs:** `metrics.json` per run, a generated HTML report, and a live stream to the dashboard.
- **Regression gate:** CI replays 200 frames of the validation sequence; fails if latency rises > 10%, memory rises at all, or mIoU drops > 1 point.
- **Unit tests for the grid:** boundary points (x exactly at 10.00 m), negative coordinates, scroll across the world origin, mip-up sums, zero-point cells.

11. Tech stack and key decisions

Layer	Choice	Alternative kept open
Language	Python 3.11 for orchestration; CUDA / C++ kernels for projection	Pure PyTorch scatter ops for v0
Deep learning	PyTorch 2.x, mixed precision	—
Sparse conv	spconv 2.x or TorchSparse	MinkowskiEngine
Point ops	torch-scatter, PyTorch3D (kNN for PointNet++)	Open3D
Inference	ONNX → TensorRT FP16	TorchScript
Middleware	ROS 2 Humble, <code>grid_map</code> export	ZeroMQ for ROS-free demo
Dashboard	React + deck.gl / three.js (WebGL), WebSocket bridge (FastAPI)	Rerun or Foxglove for quick debugging
Experiments	Hydra configs, Weights & Biases or MLflow	TensorBoard
Packaging	Docker, NVIDIA Container Toolkit	Conda env

Architecture decision records

#	Decision	Reason	Trade-off accepted
ADR-1	Square tiers, not polar or circular rings	Exact tiling of a Cartesian lattice; trivial indexing	Corners of the square extend to 141 m, slightly beyond 100 m

#	Decision	Reason	Trade-off accepted
ADR-2	Dense arrays per tier, not a quadtree or hash map	Constant-time lookup, coalesced GPU access, predictable memory	Stores the 1% inner hole of each tier
ADR-3	Integer cell-size ratios with a shared world origin	Guarantees no misalignment or double counting	Tier sizes cannot be arbitrary (5 / 50, not 5 / 45)
ADR-4	Sparse conv U-Net as default backbone	Best accuracy per millisecond on GPU for large outdoor scans	Harder dependency install than PointNet++
ADR-5	Motion from stacked past sweeps inside the network	One model, no separate tracker in v1	Weaker velocity estimates than a tracker
ADR-6	Visualiser in a separate process	Rendering can never slow the measured pipeline	Extra serialisation, about 3 ms

12. Evolution roadmap

The map message and the tier rule stay fixed; each horizon adds capability behind them.

1. **v1 — Foveated semantic map (8 weeks).** Lidar-only, 2 or 3 fixed tiers, sparse-conv segmentation with motion head, desktop GPU, dashboard and benchmark.
2. **v2 — Adaptive fovea.** Tier radii that stretch with vehicle speed (a longer fine zone at 50 km/h) and along the planned path; object tracking with IDs and velocities; Jetson Orin port with INT8.
3. **v3 — Multi-sensor and learned costs.** Camera and radar fusion into the same cells; traversability cost learned from driving logs; tiers chosen per cell by expected information rather than distance alone.